

베타테스트 자동화 도구 시스템 아키텍처 상세 설계서

System Architecture Design Document

구현 계획서 v2.0의 후속 설계 문서
다이어그램 · 컴포넌트 명세 · 데이터 모델 · 인터페이스 계약

작성일: 2026년 5월 19일 | 버전: 1.0

0. 문서 개요

0.1 목적 및 범위

이 문서는 베타테스트 자동화 도구의 시스템 아키텍처를 상세히 기술한다. 구현 계획서 v2.0 이 '무엇을 언제 만들지'에 답한다면, 이 문서는 '어떻게 구조화하고 어떻게 연결할지'에 답한다.

문서 대상은 다음과 같다.

- 현재의 1인 개발자: 구현 시 참조용
- 향후 합류할 개발자: 시스템 이해의 출발점
- 코드 리뷰어: 설계 의도 확인용

0.2 아키텍처 스타일

이 시스템은 다음 두 가지 스타일을 결합한다.

- 이벤트 기반(Event-Driven): GitHub Webhook 을 핵심 트리거로 사용. 컴포넌트 간 직접 호출 의존성을 최소화
- 포트-어댑터(Hexagonal): 외부 시스템(GitHub, LLM, Sheet) 연동은 모두 인터페이스(ABC)로 추상화. 교체·테스트가 쉬움

0.3 품질 목표 (Quality Attributes)

품질 목표	측정 가능 기준	달성 수단
응답성	폼 제출 → Issue 등록 ≤ 3 초 이슈 등록 → PR 생성 ≤ 5 분	동기 흐름은 API 응답 빠르게, 비동기 흐름은 BackgroundTask
신뢰성	Webhook 누락률 < 1% 일일 보고 누락률 < 0.5%	역등 설계 + 일일 17 시 전수 동기화로 보강
확장성	Phase 2~4 단계적 분리 가능	서비스 인터페이스(ABC) + 모노레포 구조

품질 목표	측정 가능 기준	달성 수단
관측성	모든 이벤트 추적 가능 실패 100% 알림	structlog JSON 로그 + EventLog 테이블 + Slack 알림
보안	키 노출 0 건 외부 접근 차단	Secret Manager + 사내 SSO + Webhook 서명 검증

0.4 문서 구성

- 1 장: 레벨 1 시스템 컨텍스트 — 외부 행위자와 시스템의 경계
- 2 장: 레벨 2 컨테이너 다이어그램 — 4 개 레이어 구조
- 3 장: 컴포넌트 상세 명세 — 각 컴포넌트의 책임·입출력·실패 시나리오
- 4 장: 데이터 모델 — 엔티티 관계도와 필드 명세
- 5 장: 인터페이스 계약 — REST API, Webhook 페이로드, 서비스 ABC 시그니처
- 6 장: 동적 흐름 — 시퀀스 다이어그램과 상태 머신
- 7 장: 횡단 관심사 — 인증, 로깅, 에러 처리, 멍등성
- 8 장: 배포 토폴로지 — 환경 구성, 네트워크, Secret 관리

1. 레벨 1: 시스템 컨텍스트

시스템의 가장 바깥쪽 경계를 정의한다. 베타테스트 자동화 도구는 다음 외부 행위자/시스템과 상호작용한다.

1.1 외부 행위자 (Human Actors)

행위자	주 인터페이스	시스템과의 상호작용
베타 테스터	웹 폼	사내 SSO 로 로그인 → 폼 작성 → 제출 → 결과 페이지에서 Issue 번호 확인
개발자	GitHub	자동 생성된 PR 리뷰 → 수정 또는 머지 → 이슈 자동 종료
운영자	Google Sheet	Raw Issues 시트의 수작업 컬럼(L~P) 채우기, Dashboard 로 통계 확인, Slack 에서 일일 보고 수신

1.2 외부 시스템 (External Systems)

외부 시스템	통신 방식	역할
GitHub	REST API + Webhook	Issue/PR 의 단일 진실 출처. Webhook 으로 이벤트 푸시
Claude API	HTTPS (SDK)	이슈 분석 + diff 생성. JSON 응답 강제
Google Sheet	Sheets API v4	시각화 미리. Raw Issues / Daily Snapshot / Dashboard 시트
Slack	Incoming Webhook	운영 알림 채널 (성공·실패 모두)
사내 SSO	OAuth 2.0 / OIDC	베타 테스터 인증 + 사번/부서 정보 제공

2. 레벨 2: 컨테이너 구조

시스템 내부를 4 개 레이어로 구분한다. 각 레이어는 명확한 책임을 가지고, 상위 레이어는 하위 레이어만 호출한다 (단방향 의존성).

2.1 레이어 구조

레이어	주요 구성 요소	책임 및 의존성 규칙
Frontend	Next.js Web Form	UI 렌더링, 입력 검증, SSO 토큰 처리. Backend API 만 호출
API	FastAPI routers	HTTP 엔드포인트, 인증, 요청 검증. Service Layer 만 호출
Services	Issue / LLM / Sheet / PR / Sync	비즈니스 로직. Adapter Layer 만 호출. 외부 시스템을 직접 알지 못함
Adapters	GitHub / LLM / Sheet / Slack	외부 시스템과의 실제 통신. Service 의 ABC 인터페이스를 구현

참고. Service Layer 는 GitHub, Claude, Google Sheet 같은 구체적인 외부 시스템을 직접 import 하지 않는다. 모두 Adapter 의 ABC 를 의존성 주입(DI)으로 받는다. 이 원칙 덕분에 (1) 단위 테스트에서 Mock 으로 교체, (2) Phase 2 에서 별도 Worker 로 분리, (3) LLM Provider 교체가 모두 단순해진다.

2.2 디렉토리 구조 매핑

위 레이어가 실제 코드 디렉토리와 어떻게 매핑되는지 정리한다.

레이어	디렉토리	주요 파일
Frontend	apps/web/	app/(forms)/bug-report/page.tsx, components/IssueForm.tsx
API	apps/api/src/api/	issues.py, webhooks.py, admin.py, deps.py
Services	apps/api/src/services/	issue/, llm/, sheet/, pr/, sync/
Adapters	apps/api/src/adapters/	github_adapter.py, anthropic_adapter.py, sheets_adapter.py
Models	apps/api/src/models/	bug_report.py, issue_metadata.py, llm_response.py
Core	apps/api/src/core/	config.py, logging.py, auth.py, exceptions.py

3. 컴포넌트 상세 명세

Service Layer 의 5 개 컴포넌트와 Adapter Layer 의 4 개 컴포넌트를 각각 명세한다.

3.1 IssueService

IssueService

책임

- Pydantic 모델(BugReport, EnhancementRequest) → GitHub Issue Body 마크다운 변환, GitHub Issue 생성, 라벨 자동 부여

입력

- BugReport 또는 EnhancementRequest 인스턴스 + 작성자 메타데이터(IssueMetadata)

출력

- 생성된 GitHub Issue Number + URL

의존성

- GitHubAdapter (ABC: GitHubPort)

실패 시나리오 / 에러 처리

- Issue 생성 실패 시 GithubApiError 발생 → API 라우터에서 503 응답. 이미지 업로드 실패는 본문에 경고 표시 후 진행

LLMService

책임

- 이슈 본문 파싱, 코드 컨텍스트 수집, Claude API 호출, JSON 응답 검증, diff dry-run

입력

- GitHub Issue Number, Issue Body (마크다운)

출력

- LLMAnalysisResult (cause, hypothesis, patch, confidence, files_to_change)

의존성

- GitHubAdapter (코드 fetch), LLMProvider (ABC: LLMPort), DryRunService

실패 시나리오 / 에러 처리

- JSON 파싱 실패 시 1 회 재시도. confidence < 0.5 또는 diff 적용 불가 시 LLMNotApplicableError 발생. API 한도 초과 시 지수백오프 후 needs-retry 라벨

PRService

책임

- LLM 분석 결과를 받아 새 브랜치 생성, diff 적용, 커밋, PR 생성. PR 본문에 정형 양식 적용

입력

- GitHub Issue Number, LLMAalysisResult

출력

- 생성된 PR Number + URL

의존성

- GitHubAdapter (브랜치/커밋/PR 생성)

실패 시나리오 / 에러 처리

- diff 적용 실패 시 PRCreationError. 브랜치명 중복 시 -2, -3 접미사로 재시도. 모두 실패 시 이슈에 코멘트만 첨부 + auto-pr-failed 라벨

SheetService

책임

- Google Sheet 의 Raw Issues / Daily Snapshot 시트에 행 추가.갱신. 행 인덱스 메모리 캐시 관리

입력

- Issue 데이터 또는 PR 상태 변경 이벤트

출력

- Sheet 갱신 결과 (성공/실패 + 행 인덱스)

의존성

- SheetsAdapter (ABC: SheetPort)

실패 시나리오 / 에러 처리

- Sheets API 429 시 지수백오프. 5 회 재시도 실패 시 SheetSyncError 발생 → Slack 알림. 일일 동기화에서 보강

SyncService

책임

- 매일 17 시 KST 에 GitHub 의 모든 Issue/PR 을 페이지네이션 조회 → Raw Issues 시트와 비교 → 누락/불일치 갱신. Daily Snapshot 1 행 추가

입력

- (없음, 스케줄러에서 호출)

출력

- 동기화 결과 (추가 N 건, 갱신 M 건, 실패 K 건)

의존성

- GitHubAdapter, SheetsAdapter, SlackAdapter

실패 시나리오 / 에러 처리

- 전체 실패 시 Slack 긴급 알림 + 1 회 자동 재시도. 부분 실패는 로그 + Slack 요약

3.2 Adapter 명세

Adapter	구현 ABC (Port)	주요 메서드
GitHubAdapter	GitHubPort	create_issue, update_issue, add_labels, create_branch, commit_file, create_pr, fetch_file, list_issues, verify_webhook
AnthropicAdapter	LLMPort	analyze (issue_context, code_context) → LLMResponse
SheetsAdapter	SheetPort	append_row, update_cells, find_row_by_issue, batch_update
SlackAdapter	NotifierPort	send_message (level, title, body, fields)

4. 데이터 모델

시스템에는 7 개의 주요 엔티티가 있다. 그중 BUG_REPORT, ISSUE_METADATA 는 입력 데이터, GITHUB_ISSUE/GITHUB_PR 은 외부 시스템 데이터, SHEET_ROW 와 DAILY_SNAPSHOT 은 미러 데이터, EVENT_LOG 는 감사용 내부 데이터다.

4.1 엔티티 정의

BUG_REPORT (입력 — 폼)

베타 테스터가 폼에서 입력하는 버그 제보 데이터. 개선 사항은 EnhancementRequest 로 별도 모델.

필드	타입	필수	설명
test_account	str	Y	테스트한 계정 ID
test_performed_at	datetime	Y	접속 시간 (사용자 입력)
test_area	enum	Y	Research / Document / etc.
cause_type	enum	Y	Backend / Prompting / Cache / etc.
cause_detail	str	Y	원인 상세 단문
severity	enum	Y	치명적/높음/중간/낮음 → P1~P4
test_env	object	Y	OS, 브라우저, 디바이스, 네트워크
scenario	object	Y	세부 기능 + 시나리오 설명
reproduction	object	Y	빈도(enum) + 단계(list[str])
expected_vs_actual	object	Y	예상/실제 멀티라인
input_output	object	N	입력/실제 출력/기대 출력
screenshots	list[file]	N	최대 5 장
additional_notes	str	N	자유 기재

ISSUE_METADATA (자동 캡처 — SSO + 시스템)

폼 자체에는 없지만 시스템이 자동으로 캡처하는 메타데이터. 작성자 식별과 SLA 측정에 사용.

필드	타입	출처 및 용도
tester_id	str	SSO. 사번 (예: id1234)
tester_name	str	SSO. 이름
tester_dept	str	SSO. 부서/팀
tester_email_masked	str	SSO 에서 추출 후 마스킹 (gh.kim@***)
submitted_at	datetime	시스템 시각. Backend 가 폼 수신 시점
source	str	'web-form-v1' 등 진입점 식별

참고. 개인정보 보호: 이메일은 마스킹 후 저장. 사번은 사내에서만 의미가 있어 라벨로 사용 가능.
베타 종료 후 사번 → 익명 ID 매핑 1 회성 진행.

GITHUB_ISSUE / GITHUB_PR (외부 — GitHub)

GitHub 의 원본 데이터. 시스템은 이를 정본으로 간주하고 시트에 미러링한다.

- GITHUB_ISSUE: number(PK), title, body, labels[], state, created_at, closed_at, user_login
- GITHUB_PR: number(PK), linked_issue_number(FK), branch, state, created_at, merged_at, llm_confidence

SHEET_ROW (미러 — Google Sheet)

Raw Issues 시트의 한 행. 자동 컬럼은 시스템이 갱신하고, 수작업 컬럼(L~P, S)은 운영자가 채운다.

열	필드	관리	값 출처
A	registrant	자동	ISSUE_METADATA.testers_name
B	team	자동	ISSUE_METADATA.testers_dept
C	test_account	자동	BUG_REPORT.test_account
D	test_area	자동	BUG_REPORT.test_area
E	sub_feature	자동	BUG_REPORT.scenario.feature
F	screen	자동	BUG_REPORT.scenario.url_path
G	scenario	자동	BUG_REPORT.scenario.description

열	필드	관리	값 출처
H	symptom	자동	BUG_REPORT.cause_detail
I	reproducible	자동	Yes/No 추정
J	frequency	자동	BUG_REPORT.reproduction.frequency
K	priority	자동	severity → P1~P4 매핑
L	cause_type	수작업	운영자 분석
M	cause_detail	수작업	운영자 분석
N	assignee	수작업	운영자 배정
O	action	수작업	운영자 입력
P	action_date	수작업	운영자 입력
Q	resolved_at	자동	GITHUB_PR.merged_at
R	status	자동	Open/진행중/완료
S	deployment	수작업	운영자 입력
T	registered_at	자동	GITHUB_ISSUE.created_at
U	issue_number	자동	GITHUB_ISSUE.number (PK)
V	issue_url	자동	HYPERLINK 함수
W	pr_number	자동	GITHUB_PR.number
X	pr_status	자동	GITHUB_PR.state
Y	pr_url	자동	HYPERLINK 함수
Z	auto_pr_result	자동	success/failed/skipped
AA	days_to_resolve	수식	=IF(Q,Q-T,)
AB	test_env	자동	BUG_REPORT.test_env 요약
AC	last_sync_at	자동	Sheet 마지막 갱신 시각

DAILY_SNAPSHOT (시계열 — Google Sheet)

매일 17 시에 1 행 추가되는 통계 스냅샷. Dashboard 차트의 데이터 소스.

- snapshot_date (PK), total_issues, open_count, p1_open, p2_open, p3_open, p4_open
- closed_today, avg_resolve_days, auto_pr_attempts, auto_pr_success, active_testers_7d

EVENT_LOG (감사 — SQLite, W4 도입)

모든 이벤트의 감사 기록. 디버깅과 SLA 측정에 사용. W4 에서 SQLite 로 도입, Phase 3 에 PostgreSQL 이전.

- id (UUID, PK), event_type, issue_number, pr_number, occurred_at, status, error_message, payload(JSON)
- event_type 예시: issue_created, llm_called, llm_failed, pr_created, sheet_synced, daily_report_done

4.2 엔티티 관계

관계 핵심 요약:

- BUG_REPORT 1:1 ISSUE_METADATA (품 제출 시 함께 생성)
- BUG_REPORT 0:1 GITHUB_ISSUE (실패 시 Issue 미생성)
- GITHUB_ISSUE 0:1 GITHUB_PR (LLM 실패 시 PR 미생성, Fixes #N 으로 연결)
- GITHUB_ISSUE 1:1 SHEET_ROW (Issue 번호가 PK)
- SHEET_ROW N:1 DAILY_SNAPSHOT (날짜로 집계)
- GITHUB_ISSUE 1:N EVENT_LOG (이슈당 여러 이벤트 발생)

4.3 데이터 보관 정책

데이터	보관 기간	정책
GitHub Issue/PR	영구	GitHub 에 남김. 베타 종료 후 archive label 부여
Sheet Raw Issues	1 만 행까지	초과 시 분기별 아카이브 시트로 이전
Daily Snapshot	영구	시계열 분석용. 행 수가 적어 부담 없음
Event Log	90 일	90 일 이전 행은 야간 배치로 삭제. 중요 이벤트는 별도 archive

데이터	보관 기간	정책
개인정보 (이메일·사번)	베타 종료 시	종료 후 사번 → 익명 ID 매핑. 매핑 테이블은 안전한 곳에 보관 후 정해진 기간 후 삭제

5. 인터페이스 계약

시스템 내부와 외부 인터페이스를 모두 명세한다. 향후 팀이 합류하거나 서비스를 분리할 때 이 계약이 변경되지 않도록 안정성을 우선한다.

5.1 REST API (Frontend → Backend)

Method	Path	요청 Body	응답
POST	/api/v1/issues/bug	BugReport + multipart	201, {issue_number, url}
POST	/api/v1/issues/enhancement	EnhancementRequest	201, {issue_number, url}
POST	/api/v1/webhooks/github	GitHub Webhook payload	200, {received}
POST	/api/v1/admin/sync	(없음)	202, {job_id}
GET	/api/v1/health	(없음)	200, {status:ok}

5.1.1 인증 방식

- POST /api/v1/issues/*: 사내 SSO 토큰 필수. Authorization: Bearer <token>
- POST /api/v1/webhooks/github: X-Hub-Signature-256 헤더로 HMAC 검증
- POST /api/v1/admin/sync: 별도 internal API 키 (Cloud Run 의 IAM 인증으로 GitHub Actions 만 호출 가능)
- GET /api/v1/health: 인증 없음 (Cloud Run 의 헬스체크용)

5.1.2 에러 응답 형식

모든 에러는 다음 통일된 형식으로 응답.

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "test_account is required",
    "details": {...},
    "request_id": "req_abc123"
  }
}
```

정의된 에러 코드:

- VALIDATION_ERROR (400): 입력값 검증 실패

- AUTHENTICATION_ERROR (401): SSO 토큰 없거나 만료
- AUTHORIZATION_ERROR (403): 권한 없음
- RATE_LIMITED (429): 요청 한도 초과
- GITHUB_API_ERROR (502): GitHub API 호출 실패
- SHEET_API_ERROR (502): Sheets API 호출 실패
- LLM_API_ERROR (502): Claude API 호출 실패
- INTERNAL_ERROR (500): 그 외 모든 예외

5.2 GitHub Webhook 페이로드

시스템이 구독하는 이벤트와 페이로드의 핵심 필드만 정리한다 (전체 페이로드는 GitHub 공식 문서 참조).

이벤트	action	주요 페이로드 필드
issues	opened	issue.number, issue.title, issue.body, issue.labels[], issue.user.login
issues	closed/edited/labeled	issue.number, issue.state, changes
pull_request	opened	pull_request.number, body (Fixes #N 추출), head.ref
pull_request	closed (merged)	pull_request.merged, pull_request.merged_at

5.3 LLM API 인터페이스 (LLMPort ABC)

LLMService 는 LLMPort 인터페이스만 의존한다. AnthropicAdapter 가 실제 구현. 향후 다른 LLM Provider 로 교체 가능.

Port 메서드 시그니처 (개념):

```
class LLMPort(ABC):
    async def analyze(
        issue: IssueContext,
        code_files: list[CodeFile],
        max_tokens: int = 8000
    ) -> LLMAalysisResult: ...
```

LLMAalysisResult 스키마:

필드	타입	설명
cause	str	원인 1~2 문장
hypothesis	str	패치가 왜 해결하는지
patch	str	unified diff (git apply 호환)
files_to_change	list[str]	변경되는 파일 경로
confidence	float	0~1. 0.5 미만 시 PR 미생성
review_checklist	list[str]	사람 리뷰어 체크리스트

5.4 Sheet API 인터페이스 (SheetPort ABC)

주요 메서드 시그니처:

```
class SheetPort(ABC):
    def append_row(sheet: str, values: list[Any]) -> int: ...
    def update_cells(sheet: str, range: str, values: list[list]) -> None: ...
    def find_row_by_issue(issue_number: int) -> int | None: ...
    def batch_update(updates: list[CellUpdate]) -> None: ...
```

참고. 실제 Google Sheets API 는 1-based 인덱싱과 A1 표기법을 사용한다. SheetPort 는 이를 내부에 캡슐화하여 호출 코드가 알 필요 없도록 한다.

5.5 Issue Body 마크다운 포맷 (Internal Contract)

IssueService 가 생성하는 Issue Body 의 표준 형식. 이 형식은 LLMService 가 파싱할 때도 사용되므로 변경 시 양쪽 모두 영향.

```
---
tester_id: id1234
tester_name: 김관호
tester_dept: neolab
test_account: sarah1
submitted_at: 2026-05-19T16:08:23+09:00
test_performed_at: 2026-05-19T16:05:00+09:00
---

## Summary
- 테스트 계정: ...
- 접속 시간: ...
- 테스트 영역: ...
- 원인 유형: ...
- 원인 상세: ...
- 심각도: ...
```

- 테스트 환경:

- OS: ...
- 브라우저: ...
- 디바이스: ...
- 네트워크: ...

테스트 시나리오 / 재현 방법 / 예상·실제 / 화면 / 입출력 / 추가 의견
(기존 템플릿 그대로)

6. 동적 흐름

6.1 시퀀스: 폼 제출 → Issue 등록

동기 흐름이다. API 응답을 3 초 이내에 반환해야 사용자 경험이 좋다.

1. Frontend 가 SSO 토큰을 헤더에 담아 POST /api/v1/issues/bug 호출
2. API 라우터가 토큰 검증 + 사번/부서 추출
3. Pydantic 검증으로 BugReport 모델 생성
4. IssueService.create_issue() 호출
5. IssueService 가 마크다운 Body 생성 + 라벨 결정 + GitHubAdapter.create_issue() 호출
6. GitHubAdapter 가 PyGithub 로 Issue 생성
7. Issue 번호와 URL 이 응답으로 반환

6.2 시퀀스: Webhook 수신 → PR 자동 생성

비동기 흐름. FastAPI BackgroundTasks 로 처리. Webhook 은 200 OK 를 빠르게 반환해야 GitHub 가 재시도를 시작하지 않는다.

8. GitHub 가 issues.opened 이벤트를 POST /api/v1/webhooks/github 에 발송
9. API 라우터가 X-Hub-Signature-256 검증 후 즉시 200 OK 응답
10. BackgroundTask 로 WebhookHandler.handle_issue_opened() 실행
11. SheetService.append_row()로 Raw Issues 에 행 추가 (자동 컬럼만)
12. LLMService.analyze() 호출
13. LLMService 내부: 코드 컨텍스트 수집 → Claude API 호출 → JSON 검증 → dry-run
14. 성공 시 PRService.create_pr()로 브랜치+커밋+PR 생성
15. SheetService.update_cells()로 시트의 W·X·Y·Z 컬럼 갱신
16. EventLog 에 모든 단계 기록

6.3 시퀀스: 매일 17 시 동기화

17. GitHub Actions cron 이 POST /api/v1/admin/sync 호출 (IAM 인증)
18. API 라우터가 SyncService.run_daily_sync()를 BackgroundTask 로 시작
19. SyncService 가 GitHub Issues/PRs 페이지네이션 조회 (state=all, since=어제 17 시)
20. 기존 Sheet 행과 비교 → 누락은 추가, 불일치는 갱신 (자동 컬럼만)

- 21. Daily Snapshot 통계 계산 후 1 행 추가
- 22. SlackAdapter.send_message()로 결과 요약 알림

6.4 Issue 상태 머신

From	Event	To	Side Effect
(none)	폼 제출	Open	Sheet 행 추가, LLM 트리거
Open	PR 생성됨	Open (auto-pr-created)	Sheet W·X·Y·Z 갱신
Open (auto-pr-created)	PR 머지	Closed	Sheet R=완료, Q=오늘
Open	LLM 실패	Open (auto-pr-failed)	Sheet Z=failed, 코멘트 첨부
Open	수동 close	Closed	Sheet R=완료

7. 횡단 관심사

7.1 인증 및 인가

- Frontend ↔ Backend: 사내 SSO (OIDC). JWT 토큰을 Authorization 헤더에
- Backend ↔ GitHub: GitHub App 의 짧은 수명 토큰 (Installation Token, 1 시간 유효)
- Backend ↔ Claude: ANTHROPIC_API_KEY (Secret Manager)
- Backend ↔ Sheet: 서비스 계정 JSON 키 (Secret Manager)
- Webhook 수신: HMAC-SHA256 서명 검증 (X-Hub-Signature-256)
- 관리자 엔드포인트: Cloud Run IAM 으로 GitHub Actions 만 호출 가능하게 제한

7.2 로깅 및 관측성

- structlog 로 JSON 구조화 로그 생성. Cloud Logging 에 자동 수집
- 모든 로그에 trace_id 포함 (요청-응답 추적)
- EventLog 테이블에 비즈니스 이벤트 기록 (Audit Trail)
- 핵심 SLI 지표: 품 제출 응답시간, 이슈→PR 생성 시간, LLM 머지율, 일일 동기화 성공률
- Sentry 로 에러 자동 알림 (Phase 2 이후)

7.3 멍등성 (Idempotency)

Webhook 은 GitHub 가 재시도할 수 있으므로 같은 이벤트가 두 번 도착할 수 있다. 시스템은 다음 원칙으로 멍등성을 보장한다.

- SheetService.append_row(): 이슈 번호로 기존 행 검색. 있으면 update, 없으면 append
- PRService.create_pr(): 같은 이슈 번호에 대해 이미 PR 있으면 새로 만들지 않음
- LLMService.analyze(): 같은 이슈에 대해 EventLog 에 llm_called 기록 있으면 skip (옵션)
- 브랜치명 충돌 시 fix/issue-N-2, -3 으로 자동 증분

7.4 에러 처리 전략

- Adapter 레벨: 재시도(지수백오프) + 도메인 예외로 변환 (GithubApiError 등)
- Service 레벨: 비즈니스 예외 발생 (LLMNotApplicableError, PRCreationError 등)

- API 레벨: 예외 → HTTP 에러 응답 매핑 (FastAPI exception_handlers)
- Webhook 핸들러: 절대 예외 전파하지 않음. 모든 예외 catch + 로그 + EventLog 기록

7.5 설정 관리

- 환경변수: pydantic-settings 로 검증 + 타입 안전성
- Secret: Cloud Run 에서 Secret Manager 자동 마운트
- 환경별 설정: dev / staging / prod 분리 (Cloud Run 리비전 라벨 활용)
- Feature Flag: 단순 환경변수로 시작 (LLM_ENABLED, AUTO_PR_ENABLED 등)

7.6 비밀번호 회전

- GitHub App Private Key: 6 개월마다 재발급
- Claude API Key: 분기마다 회전
- 서비스 계정 키: 분기마다 회전 (Workload Identity 로 키리스 전환 검토)
- Webhook Secret: 6 개월마다 재생성
- 모든 회전 절차는 운영 매뉴얼에 명시

8. 배포 토폴로지

8.1 환경 구성

환경	Backend	Frontend	GitHub 저장소
로컬	uvicorn (Docker)	next dev	dev-issue-track (테스트용)
Staging	Cloud Run staging	Vercel preview branch	staging-issue-track
Production	Cloud Run main	Vercel production	SIZL-Agentic-Brain-Issue-Track

8.2 네트워크 및 접근 제한

- Frontend (Vercel): 사내 IP 대역에서만 접근 가능 (Vercel Edge Middleware)
- Frontend → Backend: HTTPS over public internet, but JWT 검증
- Backend (Cloud Run): public ingress 허용하되 SSO/IAM/Webhook 서명 중 하나 필수
- Backend → GitHub/Claude/Sheet: outbound HTTPS
- GitHub Webhook → Backend: 공개 엔드포인트지만 HMAC 서명 검증 필수
- GitHub Actions cron → Backend admin: Cloud Run IAM (service account)

8.3 Secret 관리

모든 Secret 은 Google Secret Manager 에 저장하고 Cloud Run 컨테이너에 자동 마운트한다. 로컬 개발은 .env.local (gitignore).

Secret 이름	타입	사용처
GITHUB_APP_ID	string	GitHub App 식별자
GITHUB_APP_PRIVATE_KEY	PEM	GitHub App 인증
GITHUB_WEBHOOK_SECRET	random string	Webhook 서명 검증
ANTHROPIC_API_KEY	API key	Claude API 호출
GOOGLE_SA_JSON	JSON	Sheets API 인증

Secret 이름	타입	사용처
GOOGLE_SHEET_ID	string	대상 Spreadsheet
SLACK_WEBHOOK_URL	URL	Slack 알림
SSO_OIDC_CONFIG	JSON	SSO 검증 (issuer, jwks_uri)
INTERNAL_API_KEY	random string	admin/sync 엔드포인트 (대체)

8.4 배포 파이프라인

23. 개발자가 PR 생성 → CI 워크플로우 실행 (lint + mypy + test)
24. PR 머지 → main 브랜치 푸시
25. deploy-api.yml: Docker 빌드 → Cloud Run 에 배포 (staging 먼저, 검증 후 prod)
26. deploy-web.yml: Vercel 이 자동 감지 → 빌드 → 배포
27. 배포 완료 후 health check + Slack 알림

8.5 스케일링 정책

- Cloud Run min instances: 1 (콜드 스타트 회피)
- Cloud Run max instances: 10 (예상치 못한 부하 대비)
- 동시 요청 처리: instance 당 80 (FastAPI 비동기)
- Phase 2 도래 시 LLM Worker 를 별도 Cloud Run 서비스로 분리

8.6 모니터링

- Cloud Run 기본 메트릭: 요청 수, p95 응답시간, 에러율, instance 수
- 커스텀 메트릭: LLM 호출 비용, 머지율, 일일 동기화 소요시간
- 알람: 5xx 비율 > 5%, p95 응답시간 > 3 초, LLM 일일 비용 > 한도의 80%
- Slack: 모든 알람 + 일일 보고 결과

9. 부록

9.1 약어

- ABC: Abstract Base Class (Python 의 추상 인터페이스)
- ADR: Architecture Decision Record
- DI: Dependency Injection
- DTO: Data Transfer Object
- ERD: Entity Relationship Diagram
- MVP: Minimum Viable Product
- PII: Personally Identifiable Information
- SLA/SLI: Service Level Agreement / Indicator
- SoT: Source of Truth
- SSO: Single Sign-On

9.2 관련 문서

- 구현 계획서 v2.0 (베타테스트_도구_구현계획서_v2.0.docx)
- PRD (별도 작성 시)
- 운영 매뉴얼 (W4 산출물)

9.3 변경 이력

- v1.0 (2026-05-19): 초안 작성. 4 개 레이어, 5 개 Service, 4 개 Adapter 구조 확정

9.4 향후 보완 예정 항목

- ADR 1~8 정식 문서화 (W0 종료 후)
- Phase 2 분리 시 LLM Worker 통신 프로토콜 상세 (Redis Pub/Sub vs Celery)
- 멀티테넌시 도입 시 데이터 격리 전략 (Phase 4)
- 부하 테스트 결과 및 스케일링 임계값 (베타 종료 후)